

Deep Learning Winter term 25/26 – Exercise Sheet 7

Prepared by: Ashik Emon

1. Momentum (4P)

a) Two Iterations of Gradient Descent with Momentum

Given:

$$f(\theta) = \frac{\theta^3}{3} + 20\theta^2 - 50\theta - 5$$

with learning rate $\alpha = 0.01$, momentum $\phi = 0.5$, initial velocity $v_0 = 0$, and starting point $\theta_0 = 20$.

Step 1: Compute the derivative

$$f'(\theta) = \frac{d}{d\theta} \left(\frac{\theta^3}{3} + 20\theta^2 - 50\theta - 5 \right) = \theta^2 + 40\theta - 50$$

Step 2: Iteration formula

$$v_t = \phi v_{t-1} - \alpha f'(\theta_{t-1})$$

$$\theta_t = \theta_{t-1} + v_t$$

Iteration 1:

$$f'(20) = 20^2 + 40 \times 20 - 50 = 400 + 800 - 50 = 1150$$

$$v_1 = 0.5 \times 0 - 0.01 \times 1150 = -11.5$$

$$\theta_1 = 20 + (-11.5) = 8.5$$

Iteration 2:

$$f'(8.5) = (8.5)^2 + 40 \times 8.5 - 50 = 72.25 + 340 - 50 = 362.25$$

$$v_2 = 0.5 \times (-11.5) - 0.01 \times 362.25 = -5.75 - 3.6225 = -9.3725$$

$$\theta_2 = 8.5 + (-9.3725) = -0.8725$$

Result after 2 iterations with momentum:

$$\theta_2 \approx -0.87$$

b) Benefit of Using Momentum vs. Vanilla Gradient Descent

Vanilla Gradient Descent:

$$\theta_t = \theta_{t-1} - \alpha f'(\theta_{t-1})$$

Iteration 1:

$$\theta_1 = 20 - 0.01 \times 1150 = 8.5$$

Iteration 2:

$$\theta_2 = 8.5 - 0.01 \times 362.25 = 8.5 - 3.6225 = 4.8775$$

Comparison:

- **Momentum:** Moves much faster, reaching -0.87 after 2 steps.
- **Vanilla GD:** Moves more slowly, reaching 4.88 after 2 steps.
- **Benefit:** Momentum accelerates convergence, especially in directions of consistent gradient, but can overshoot if not tuned.

2. Adam (4P)

Given:

- Gradients: $g_0 = (4, 2, 3, 1)^T$, $g_1 = (1, 2, 1, 2)^T$
- $\rho_1 = 0.9$, $\rho_2 = 0.99$, $\alpha = 0.01$
- $s_0 = 0$, $r_0 = 0$

Adam update rules:

$$s_t = \rho_1 s_{t-1} + (1 - \rho_1) g_t$$

$$r_t = \rho_2 r_{t-1} + (1 - \rho_2) g_t^2$$

Bias correction:

$$\hat{s}_t = \frac{s_t}{1 - \rho_1^t}, \quad \hat{r}_t = \frac{r_t}{1 - \rho_2^t}$$

Update:

$$\Delta\theta_t = -\alpha \frac{\hat{s}_t}{\sqrt{\hat{r}_t} + \epsilon}$$

Iteration 0:

$$s_0 = 0.1 \times (4, 2, 3, 1) = (0.4, 0.2, 0.3, 0.1)$$

$$r_0 = 0.01 \times (16, 4, 9, 1) = (0.16, 0.04, 0.09, 0.01)$$

Bias correction:

$$\hat{s}_0 = (4, 2, 3, 1)$$

$$\hat{r}_0 = (16, 4, 9, 1)$$

Update:

$$\Delta\theta_0 = -0.01 \frac{(4, 2, 3, 1)}{(\sqrt{16}, \sqrt{4}, \sqrt{9}, \sqrt{1})} = -0.01 \times (1, 1, 1, 1) = (-0.01, -0.01, -0.01, -0.01)$$

Iteration 1:

$$s_1 = 0.9 \times (0.4, 0.2, 0.3, 0.1) + 0.1 \times (1, 2, 1, 2) = (0.46, 0.38, 0.37, 0.29)$$

$$r_1 = 0.99 \times (0.16, 0.04, 0.09, 0.01) + 0.01 \times (1, 4, 1, 4) = (0.1684, 0.0796, 0.0991, 0.0499)$$

Bias correction:

$$\hat{s}_1 = \frac{(0.46, 0.38, 0.37, 0.29)}{1 - 0.9^2} = \frac{(0.46, 0.38, 0.37, 0.29)}{0.19} \approx (2.42, 2.00, 1.95, 1.53)$$

$$\hat{r}_1 = \frac{(0.1684, 0.0796, 0.0991, 0.0499)}{1 - 0.99^2} = \frac{(0.1684, 0.0796, 0.0991, 0.0499)}{0.0199} \approx (8.46, 4.00, 4.98, 2.51)$$

Update:

$$\begin{aligned} \Delta\theta_1 &= -0.01 \frac{(2.42, 2.00, 1.95, 1.53)}{(\sqrt{8.46}, \sqrt{4.00}, \sqrt{4.98}, \sqrt{2.51})} \\ &\approx -0.01 \times (0.83, 1.00, 0.87, 0.97) = (-0.0083, -0.01, -0.0087, -0.0097) \end{aligned}$$

Summary:

$$\begin{aligned} \Delta\theta_0 &= (-0.01, -0.01, -0.01, -0.01) \\ \Delta\theta_1 &\approx (-0.0083, -0.01, -0.0087, -0.0097) \end{aligned}$$

3. Initialization (2P)

Initializing all parameters with a small constant c is a bad idea because:

- All neurons start identically, leading to identical gradients and updates.
- No symmetry breaking, so the network cannot learn diverse features.

Better strategies:

- **Random initialization** (breaks symmetry).
- **Xavier/Glorot initialization:** $\mathcal{N}(0, \frac{1}{n_{in}})$ for tanh/linear activations.
- **He initialization:** $\mathcal{N}(0, \frac{2}{n_{in}})$ for ReLU activations.
- These maintain variance across layers and help gradients flow properly.